

Manuale: VPS Economica + Coolify — Setup Completo

Obiettivo

Installare e configurare una VPS economica con Coolify come piattaforma di deployment self-hosted per:

- AIA Platform (108 Vision) — Hono API + React Dashboard + PostgreSQL + Redis + Qdrant + Neo4j + LiteLLM
- AIA Website — Astro + TinaCMS
- WellBeingApp API — .NET 9 API + PostgreSQL + AI services (Qwen TTS/Text)

1. Scelta VPS

Provider consigliati (rapporto prezzo/performance)

Provider	Piano	vCPU	RAM	Disco	Prezzo/mese	Note
Hetzner (consigliato)	CX32	4	8 GB	80 GB NVMe	~7-8 EUR	Datacenter EU (Falkenstein/Helsinki), ottima rete
Hetzner	CX42	4	16 GB	160 GB NVMe	~14 EUR	Se servono Neo4j + Qdrant + tutti i servizi
Netcup	VPS 2000 G11	6	8 GB	256 GB	~9 EUR	Buon rapporto storage/prezzo
Contabo	Cloud VPS M	6	16 GB	400 GB	~12 EUR	Tanto storage, rete meno stabile

Requisiti minimi per il carico completo

Risorsa	Minimo	Consigliato
vCPU	4	6
RAM	8 GB	16 GB
Disco	80 GB NVMe	160 GB NVMe
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
IPv4	1 statico	1 statico

****Raccomandazione:**** Hetzner CX42 (16 GB RAM, ~14 EUR/mese) per far girare tutto senza swap pressure. Se vuoi risparmiare, CX32 (8 GB) funziona disabilitando Neo4j e limitando Qdrant a 512 MB.

2. Setup Iniziale VPS

2.1 Accesso SSH e hardening base

```
# Dal tuo PC — prima connessione
ssh root@<IP_VPS>

# Aggiorna il sistema
apt update && apt upgrade -y

# Installa strumenti base
apt install -y curl wget git ufw fail2ban htop

# Crea utente non-root
adduser deploy
usermod -aG sudo deploy

# Configura SSH key (dal tuo PC locale)
# ssh-keygen -t ed25519 -C "deploy@vps"
# ssh-copy-id deploy@<IP_VPS>

# Disabilita login root e password auth
sed -i 's/^PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/^#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart sshd
```

2.2 Firewall (UFW)

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp    # SSH
ufw allow 80/tcp    # HTTP
ufw allow 443/tcp   # HTTPS
ufw allow 8000/tcp  # Coolify UI (temporaneo, poi via reverse proxy)
ufw enable
```

2.3 Swap (se 8 GB RAM)

```
fallocate -l 4G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab

# Ottimizza swappiness per workload Docker
echo 'vm.swappiness=10' >> /etc/sysctl.conf
sysctl -p
```

3. Installazione Coolify

3.1 Installazione one-liner

```
# Come root o con sudo
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash
```

Questo installa:

- Docker Engine + Docker Compose
- Coolify (container-based, self-updating)

- Traefik come reverse proxy (gestione SSL automatica via Let's Encrypt)

3.2 Primo accesso

- Apri `http://<IP_VPS>:8000` nel browser
- Crea l'account admin (email + password)
- Configura il "Server" locale (Coolify lo rileva automaticamente)

3.3 Configurazione DNS

Punta i tuoi domini al VPS prima di procedere:

Record	Nome	Valore
A	coolify.tuodominio.it	<IP_VPS>
A	api.108vision.it	<IP_VPS>
A	dashboard.108vision.it	<IP_VPS>
A	108vision.it	<IP_VPS>
A	wellbeing-api.tuodominio.it	<IP_VPS>
A	tina.108vision.it	<IP_VPS>

3.4 Configura Coolify con dominio custom

Dalla UI Coolify → Settings:

- Instance's Domain: `https://coolify.tuodominio.it`
- Salva → Coolify genererà il certificato SSL via Let's Encrypt

Ora chiudi la porta 8000:

```
ufw delete allow 8000/tcp
```

4. Deployment: AIA Platform (108 Vision)

4.1 Architettura servizi



4.2 Setup tramite Coolify — Metodo Docker Compose

Step 1: Crea progetto in Coolify

Coolify UI → Projects → New Project → "108 Vision Platform"

Step 2: Aggiungi risorsa "Docker Compose"

New Resource → Docker Compose → inserisci il seguente compose (adattato per Coolify):

```
# docker-compose.coolify.yml — AIA Platform
version: "3.9"

services:
  # === DATABASES ===
  postgres:
    image: pgvector/pgvector:pg16
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-aia}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB:-aia_platform}
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-aia}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    restart: always
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  qdrant:
    image: qdrant/qdrant:latest
    restart: always
    environment:
      QDRANT__SERVICE__API_KEY: ${QDRANT_API_KEY}
    volumes:
      - qdrant_data:/qdrant/storage
    # Limita RAM su VPS piccola
    deploy:
      resources:
        limits:
          memory: 512M

  neo4j:
    image: neo4j:5-community
    restart: always
    environment:
      NEO4J_AUTH: neo4j/${NEO4J_PASSWORD}
      NEO4J_PLUGINS: '["apoc"]'
      NEO4J_server_memory_heap_initial__size: 256m
      NEO4J_server_memory_heap_max__size: 512m
      NEO4J_server_memory_pagecache_size: 128m
    volumes:
```

Step 3: Configura variabili ambiente in Coolify

Nella sezione "Environment Variables" del progetto.

```
# Database
POSTGRES_USER=aia
POSTGRES_PASSWORD=<genera-password-32-char>
POSTGRES_DB=aia_platform

# Redis
REDIS_PASSWORD=<genera-password-32-char>

# Qdrant
QDRANT_API_KEY=<genera-api-key>

# Neo4j
NEO4J_PASSWORD=<genera-password-32-char>

# LiteLLM
LITELLM_MASTER_KEY=sk-<genera-key-32-char>

# App
AUTH_SECRET=<genera-secret-64-char>
GITHUB_USER=EliosScoglio

# AI Provider Keys (per litellm-config.yaml)
DEEPSEEK_API_KEY=<tua-key>
DASHSCOPE_API_KEY=<tua-key>
OPENAI_API_KEY=<tua-key-opzionale>
```

Step 4: LiteLLM Config

Crea il file `litellm-config.yaml` (caricalo via Coolify file mount o Git):

```
model_list:
- model_name: fast-cheap
  litellm_params:
    model: deepseek/deepseek-chat
    api_key: os.environ/DEEPSEEK_API_KEY
- model_name: balanced
  litellm_params:
    model: deepseek/deepseek-reasoner
    api_key: os.environ/DEEPSEEK_API_KEY
- model_name: powerful
  litellm_params:
    model: dashscope/qwen-max
    api_key: os.environ/DASHSCOPE_API_KEY
- model_name: embedding
  litellm_params:
    model: dashscope/text-embedding-v3
    api_key: os.environ/DASHSCOPE_API_KEY

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL
```

4.3 Build e Deploy da Git

Alternativa: Deploy diretto da repository Git

Coolify UI → New Resource → "Application" (non Docker Compose):

- Source: Git Repository → <https://github.com/EliosScoglio/aia-platform>
- Branch: main
- Build Pack: Nixpacks (auto-detect Node.js) oppure Dockerfile

- Port: 3000

Per il monorepo, configura build separati per ogni app:

- Gateway: Build Path = apps/gateway, Dockerfile = apps/gateway/Dockerfile
- Dashboard: Build Path = apps/dashboard, Dockerfile = apps/dashboard/Dockerfile

5. Deployment: AIA Website (Astro + TinaCMS)

5.1 Opzione A: Self-hosted su Coolify (consigliato per indipendenza)

Coolify UI → New Resource → Application:

Campo | Valore

Source | Git repo URL

Branch | main

Build Pack | Nixpacks

Build Command | npx tinacms build && npx astro build

Start Command | node ./dist/server/entry.mjs

Port | 4321

Domain | 108vision.it

Variabili ambiente:

```
TINA_PUBLIC_CLIENT_ID=<tina-cloud-client-id>
TINA_TOKEN=<tina-read-token>
# oppure self-hosted TinaCMS:
TINA_PUBLIC_IS_LOCAL=false
```

5.2 Opzione B: Mantenere su Vercel (gratis, zero maintenance)

Se il sito è già su Vercel con il free tier, ha senso tenerlo lì:

- Build automatica da Git push
- CDN globale inclusa
- Zero costi per siti statici/SSR leggeri

****Raccomandazione:**** *tieni il website su Vercel (gratis, CDN globale) e usa la VPS solo per backend/databases.*

6. Deployment: WellBeingApp API

6.1 Architettura



6.2 Docker Compose per Coolify

```
# docker-compose.coolify.yml — WellBeingApp
version: "3.9"

services:
  wellbeing-api:
    build:
      context: .
      dockerfile: WellBeingApi/Dockerfile
    restart: always
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_URLS: http://+:8080
      ConnectionStrings__Postgres: "Host=wellbeing-db;Port=5432;Database=wellbeing;Username=wellbeing;Password=${WB_DB_P
      UserDataStorage__Provider: Postgres
      Jwt__Key: ${WB_JWT_KEY}
      Jwt__Issuer: https://wellbeing-api.tuodominio.it
      Jwt__Audience: WellBeingApp
      Jwt__ExpireMinutes: 1440
      QwenDashScopeTextGenerator__ApiKey: ${DASHSCOPE_API_KEY}
      QwenDashScopeTextGenerator__Model: qwen3.6-flash
      Qwen3Tts__ApiKey: ${DASHSCOPE_API_KEY}
      Google__ClientId: ${GOOGLE_CLIENT_ID}
      Facebook__AppId: ${FACEBOOK_APP_ID}
      Facebook__AppSecret: ${FACEBOOK_APP_SECRET}
      Admin__Username: ${WB_ADMIN_USER}
      Admin__Password: ${WB_ADMIN_PASSWORD}
    depends_on:
      wellbeing-db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health/live"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 45s
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.wellbeing.rule=Host(`wellbeing-api.tuodominio.it`)"
      - "traefik.http.routers.wellbeing.tls.certresolver=letsencrypt"
      - "traefik.http.services.wellbeing.loadbalancer.server.port=8080"
    volumes:
      - wb_media:/app/media:ro

  wellbeing-db:
    image: postgres:16-alpine
    restart: always
    environment:
      POSTGRES_USER: wellbeing
      POSTGRES_PASSWORD: ${WB_DB_PASSWORD}
      POSTGRES_DB: wellbeing
    volumes:
      - wb_pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U wellbeing"]
      interval: 10s
```

6.3 Dockerfile per WellBeingApi

Se non esiste già nel progetto, crealo:

```
# WellBeingApi/Dockerfile
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src

# Copia solution e progetti per restore
COPY *.sln .
COPY WellBeingApi/*.csproj WellBeingApi/
COPY WellBeing/*.csproj WellBeing/
COPY SharedStuff/*.csproj SharedStuff/
RUN dotnet restore WellBeingApi/WellBeingApi.csproj

# Copia tutto e builda
COPY . .
RUN dotnet publish WellBeingApi/WellBeingApi.csproj -c Release -o /app/publish --no-restore

# Runtime image
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS runtime
WORKDIR /app
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "WellBeingApi.dll"]
```

6.4 Variabili ambiente Coolify

```
# Database
WB_DB_PASSWORD=<genera-password-32-char>

# JWT (minimo 32 caratteri!)
WB_JWT_KEY=<genera-key-64-char-alfanumerica>

# AI Services
DASHSCOPE_API_KEY=<tua-key-alibaba-dashscope>

# Social Login
GOOGLE_CLIENT_ID=<tuo-google-oauth-client-id>
FACEBOOK_APP_ID=<tuo-facebook-app-id>
FACEBOOK_APP_SECRET=<tuo-facebook-app-secret>

# Admin
WB_ADMIN_USER=admin
WB_ADMIN_PASSWORD=<genera-password-sicura>
```

6.5 Alternativa: Database condiviso

Se vuoi risparmiare RAM, usa la stessa istanza PostgreSQL di AIA Platform:

```
ConnectionStrings__Postgres=Host=postgres;Port=5432;Database=wellbeing;Username=wellbeing;Password=<password>
```

Crea il database e l'utente:

```
CREATE USER wellbeing WITH PASSWORD '<password>';
CREATE DATABASE wellbeing OWNER wellbeing;
```

7. Configurazione Condivisa

7.1 PostgreSQL condiviso (singola istanza, multi-database)

Se usi un solo PostgreSQL per tutto:

```
-- Esegui dentro il container postgres
CREATE DATABASE aia_platform;
CREATE DATABASE wellbeing;
CREATE DATABASE litellm;

CREATE USER aia WITH PASSWORD '<pw1>';
CREATE USER wellbeing WITH PASSWORD '<pw2>';
CREATE USER litellm WITH PASSWORD '<pw3>';

GRANT ALL PRIVILEGES ON DATABASE aia_platform TO aia;
GRANT ALL PRIVILEGES ON DATABASE wellbeing TO wellbeing;
GRANT ALL PRIVILEGES ON DATABASE litellm TO litellm;

-- pgvector extension per AIA
\c aia_platform
CREATE EXTENSION IF NOT EXISTS vector;
```

7.2 Backup automatico

Crea uno script di backup giornaliero via Coolify "Scheduled Task" o cron:

```
#!/bin/bash
# /opt/backup/backup-dbs.sh
BACKUP_DIR="/opt/backup/daily"
DATE=$(date +%Y%m%d_%H%M)
mkdir -p $BACKUP_DIR

# PostgreSQL (tutti i database)
docker exec $(docker ps -qf "name=postgres") \
  pg_dumpall -U aia > "$BACKUP_DIR/pg_all_$DATE.sql"

# Comprimi e ruota (mantieni ultimi 7 giorni)
gzip "$BACKUP_DIR/pg_all_$DATE.sql"
find $BACKUP_DIR -name "*.gz" -mtime +7 -delete

# Opzionale: upload su S3/Backblaze B2
# rclone copy $BACKUP_DIR remote:backups/
```

Cron (aggiungi a /etc/crontab o via Coolify):

```
0 3 * * * root /opt/backup/backup-dbs.sh
```

7.3 Monitoring

Coolify include monitoring base. Per qualcosa in piu:

```
# Aggiungi al docker-compose principale
services:
  uptime-kuma:
    image: louislam/uptime-kuma:1
    restart: always
    volumes:
      - uptime_data:/app/data
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.uptime.rule=Host(`status.tuodominio.it`)"
      - "traefik.http.routers.uptime.tls.certresolver=letsencrypt"
      - "traefik.http.services.uptime.loadbalancer.server.port=3001"

volumes:
  uptime_data:
```

8. Stima Risorse e Costi

8.1 Consumo RAM stimato

Servizio	RAM stimata
Coolify + Traefik	~500 MB
PostgreSQL (shared)	~500 MB
Redis	~100 MB
Qdrant	~512 MB
Neo4j	~768 MB
LiteLLM	~256 MB
AIA Gateway (Node.js)	~200 MB
AIA Dashboard (Nginx)	~50 MB
WellBeingApi (.NET 9)	~300 MB
TOTALE	~3.2 GB

Con OS + buffer: ~5 GB effettivi → 8 GB funziona, 16 GB comodo.

8.2 Costo mensile totale

Voce	Costo
VPS Hetzner CX32 (8 GB)	7.50 EUR
oppure CX42 (16 GB)	14.00 EUR
Domini (se non già posseduti)	~1-2 EUR/mese ammortizzato
Backup storage (Backblaze B2)	~1 EUR
TOTALE	~9-17 EUR/mese

8.3 Configurazione "Risparmio massimo" (8 GB RAM)

Se vuoi stare su 8 GB:

- Disabilita Neo4j (usa solo se hai il knowledge graph attivo)
- Limita Qdrant a 256 MB
- Limita PostgreSQL a 256 MB shared_buffers
- Usa 4 GB swap

9. Checklist Deployment

Pre-deployment

- ☐ VPS ordinata e accessibile via SSH
- ☐ DNS propagato (verifica con `dig +short tuodominio.it`)
- ☐ SSH key configurata, root login disabilitato
- ☐ Firewall attivo (solo 22, 80, 443)
- ☐ Coolify installato e accessibile via HTTPS

AIA Platform

- ☐ Docker Compose caricato in Coolify
- ☐ Variabili ambiente configurate
- ☐ `litellm-config.yaml` montato
- ☐ Database creato con estensione pgvector
- ☐ Migrations eseguite (`npm run db:migrate` o automatico al primo avvio)
- ☐ Health check attivo: `curl https://api.108vision.it/health`
- ☐ Dashboard raggiungibile: `https://dashboard.108vision.it`

AIA Website

- ☐ Deploy su Vercel (o Coolify se preferisci self-hosted)
- ☐ TinaCMS configurato e funzionante
- ☐ Dominio `108vision.it` puntato correttamente

WellBeingApp API

- ☐ Dockerfile funzionante (testato localmente con `docker build`)
- ☐ Docker Compose caricato in Coolify
- ☐ Database PostgreSQL creato
- ☐ JWT key configurata (minimo 32 chars)
- ☐ API key Qwen/DashScope configurate
- ☐ Google OAuth Client ID configurato
- ☐ Health check attivo: `curl https://wellbeing-api.tuodominio.it/health/live`
- ☐ Test login social funzionante

Post-deployment

- ☐ Backup script configurato e testato
- ☐ Uptime monitoring attivo
- ☐ SSL certificati validi (verifica con `curl -vI https://...`)
- ☐ Coolify auto-update abilitato

10. Troubleshooting Rapido

Problema | Causa probabile | Soluzione

Container OOM killed | RAM insufficiente | Aumenta swap o riduci limiti servizi

SSL non funziona | DNS non propagato | Attendi 5-10 min, verifica con `dig`

502 Bad Gateway | Container non avviato o health check fallito | `docker logs <container>`

DB connection refused | Container DB non ready | Verifica `depends_on` + healthcheck

LiteLLM 401 | Master key errata | Verifica variabile `LITELLM_MASTER_KEY`

.NET API crash all'avvio | JWT key troppo corta | Usa minimo 32 caratteri alfanumerici

11. Comandi Utili Post-Installazione

```
# Stato servizi
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Logs di un servizio
docker logs --tail 100 -f <container_name>

# Accesso shell PostgreSQL
docker exec -it $(docker ps -qf "name=postgres") psql -U aia -d aia_platform

# Accesso Redis CLI
docker exec -it $(docker ps -qf "name=redis") redis-cli -a <password>

# Disk usage
docker system df
docker volume ls

# Pulizia immagini inutilizzate
docker image prune -a --filter "until=168h"

# Restart servizio specifico (via Coolify UI o):
docker compose -f <path> restart <service>
```

12. Upgrade e Manutenzione

Aggiornamento Coolify

Coolify si auto-aggiorna. Per forzare:

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash
```

Aggiornamento OS

```
apt update && apt upgrade -y
# Riavvio se kernel update
reboot
```

Aggiornamento applicazioni

- Via Coolify UI: abilita "Auto Deploy" sul branch main → ogni push triggera rebuild
- Manuale: Coolify UI → Application → "Redeploy"

Ultimo aggiornamento: 2026-06-12